

Inhalt

Klassen & Eigenschaften.....	0
› Lernziele.....	0
› Einstieg.....	1
› Kerninhalt.....	1
› Aufgaben.....	6
› KI-Prompts.....	7
› Weiterführend.....	7
› Reflexion.....	7

Klassen & Eigenschaften

Arbeitsblatt G1 | Modul OOP - Herbst 2026

Kopfdaten

Thema	G1 - Klassen & Eigenschaften
Voraussetzungen	keine (Einstiegsthema in Bereich 1)
Schwierigkeit	☆
Zeitaufwand	1-2 Doppelkationen
Folgethema	G2 - Methoden & Kapselung

Bevor du startest: Fülle P1 (Vor dem Start) deines Themen-Planungsdokuments aus (Vorwissen, Ziele, Vorgehen). Es dauert nur wenige Minuten und macht dein Lernen gezielter.



Lernziele

Was du nach diesem Arbeitsblatt kannst, siehst du in der folgenden Liste. Sie hilft dir, deinen Lernfortschritt selbst einzuschätzen.

Lernziele

Nach diesem Arbeitsblatt kannst du:

- › eine Klasse als Datenstruktur erstellen und Objekte (Instanzen) davon erzeugen;
- › den Unterschied zwischen Klasse und Objekt (Instanz, Entität) erklären;
- › Felder mit Zugriffstyp, Datentyp und Namen definieren und die Clean-Code-Schreibweise (PascalCase) anwenden;
- › eine eigene Klasse als Datentyp verwenden (Assoziation) und den Objekt-Initializer { } nutzen;
- › Eigenschaften (Properties) mit Getter und Setter erstellen und ihren Mehrwert (Read-Only, berechnete Werte, Datenvalidierung) erklären und anwenden.

Vertiefung / PRO (freiwillig, nicht prüfungsrelevant):

- › erkennen, dass sich Wert- und Referenztypen bei der Zuweisung unterschiedlich verhalten (z.B. zeigt buch2 = buch1 auf dasselbe Objekt; die Begriffe byValue / byReference und Seiteneffekte vertiefst du in G2);
- › Objekte im JSON-Format speichern und wieder laden.

Abgrenzung: Methoden und das benannte Prinzip der Kapselung erarbeitest du im Folgethema G2. Dieses Blatt legt das Fundament dafür.

Einstieg

Bisher hast du Daten in einzelnen Variablen gespeichert: `string titel`, `string autor`, `int seiten`. Das funktioniert für ein Buch. Aber wie speicherst du 100 Bücher mit je vier Angaben, ohne 400 einzelne Variablen zu verwalten?

Denkanstoss (zuerst selbst überlegen): Skizziere in Stichworten, wie du die Daten von 100 Büchern strukturiert ablegen würdest. Halte deine Idee kurz fest, bevor du weiterliest. Vergleiche sie am Ende mit dem, was du gelernt hast.

Die Antwort der objektorientierten Programmierung ist die **Klasse**: eine Vorlage, die zusammengehörige Daten bündelt. Aus einer Klasse erzeugst du beliebig viele **Objekte** mit jeweils eigenen Werten.

Kerninhalt

Klassen als Datenspeicher

Objekte (auch Entitäten oder Instanzen genannt) speichern ihren Zustand in Datenfeldern. Da jedes Objekt aus einer Klasse (Vorlage) erzeugt wird, legt die Klasse fest, welche Werte gespeichert werden können. Jeder Wert hat - wie bei Variablen - einen Datentyp. Beispiel Buch:

```
internal class Buch
{
    public string Titel;
    public string Autor;
    public int Seiten;
    public string ISBN;
}
```

Wir stellen fest:

- › Der Klassenname beginnt mit einem Grossbuchstaben (Clean-Code-Regel).
- › Jedes Feld (`Titel`, `Autor`, `Seiten`, `ISBN`) hat einen Zugriffstyp (`public`), einen Datentyp und einen Namen.
- › Auch die Feldnamen beginnen mit einem Grossbuchstaben (Clean-Code-Regel).

Nun kann die Klasse verwendet werden:

```
static void Main(string[] args)
{
    Buch meinLieblingsbuch = new Buch();
    meinLieblingsbuch.Titel = "Microserfs";
    meinLieblingsbuch.Autor = "Douglas Coupland";
    meinLieblingsbuch.Seiten = 300;
    meinLieblingsbuch.ISBN = "978-3-455-01173-9";

    Console.WriteLine("Titel: " + meinLieblingsbuch.Titel);
}
```

Feststellungen:

- › Aus der Klasse Buch wird ein Objekt (Instanz, Entität) `meinLieblingsbuch` erzeugt. Deklaration und Instanziierung stehen hier auf einer Zeile.
- › Auf die Felder greifst du mit `Instanzname.Feldname` zu.
- › Felder können gelesen und geschrieben werden.

Du kannst ein zweites Buch erstellen und andere Werte speichern:

```
Buch meinAeltestesBuch = new Buch();  
meinAeltestesBuch.Titel = "Gutenbergs Leiden";
```

Deshalb werden Felder auch **Instanzvariablen** genannt: Jede Instanz hat ihre eigenen Werte.

Praxis: eine Klasse in Visual Studio anlegen

Rechtsklick auf das Projekt im Projektmappen-Explorer, dann Hinzufügen -> Klasse... , einen Namen vergeben (z.B. Buch) und bestätigen. Du erhältst eine neue, leere Klassendatei.



Arbeitsanweisung

Lege die Klasse Buch an und erstelle im Hauptprogramm mindestens zwei Instanzen. Gib die Werte auf der Konsole aus.

Tipp zur KI-Nutzung: Tippe die Grundstruktur zuerst selbst, damit du den Aufbau verinnerlichst. Setze KI danach gezielt ein, um deinen Code erklären zu lassen oder Fehler zu finden. So bleibt das Verständnis bei dir - genau darauf zielt die Prüfung (ohne KI).

Felder: Zugriffstyp, Datentyp, Name

Eine Klasse mit Feldern hat den Aufbau:

```
Zugriffstyp class Klassenname  
{  
    Zugriffstyp1 Datentyp1 Name1;  
    Zugriffstyp2 Datentyp2 Name2;  
}
```

Als Zugriffstyp verwenden wir vorerst:

Zugriffstyp	Erklärung	Nutzbar bei
<code>public</code>	Sichtbar von aussen (Teil der API)	Klassen, Feldern
<code>private</code>	Nicht sichtbar von aussen	Feldern
<code>internal</code>	Klasse nicht sichtbar ausserhalb des Projekts	Klassen

Wird der Zugriffstyp nicht angegeben, gilt: bei Klassen `internal`, bei Feldern `private`.

Offizielle Dokumentation: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/accessibility-levels>

Als Datentyp sind alle bisher bekannten Typen erlaubt: primitive (integrierte) Typen, komplexe Typen, Listentypen und eigene Klassen (siehe nächster Abschnitt).

Zur Namensgebung: Jeder Name ist im selben Kontext eindeutig. Die Clean-Code-Regel verlangt für Felder und Eigenschaften die PascalCase-Schreibweise. Schreibweisen im Überblick:

Schreibweise	Beispiel	typische Verwendung
camelCase	meinBuch	lokale Variablen, Parameter
PascalCase	MeinBuch	Klassen, Felder, Eigenschaften
snake_case	mein_buch	in C# unüblich
kebab-case	mein-buch	in C# nicht möglich (Bindestrich)

Eine eigene Klasse als Datentyp (Assoziation)

Wir erweitern das Beispiel: Statt den Autor als `string` zu speichern, definieren wir eine eigene Klasse `Autor`:

```
internal class Autor
{
    public string Name;
    public DateOnly Geburtsdatum;
    public string Nationalitaet;
}
```

Diese Klasse nutzen wir nun wie einen Datentyp:

```
internal class Buch
{
    public string Titel;
    public Autor Autor;
    public int Seiten;
    public string ISBN;
}
```

Anwendung mit dem **Objekt-Initializer** `{ }`, der direkt nach der Instanziierung Werte zuweist:

```
Autor meinLieblingsautor = new Autor()
{
    Name = "Douglas Coupland",
    Nationalitaet = "CAN",
    Geburtsdatum = new DateOnly(1961, 12, 30)
};

Buch meinLieblingsbuch = new Buch();
meinLieblingsbuch.Titel = "Microserfs";
meinLieblingsbuch.Autor = meinLieblingsautor;
meinLieblingsbuch.Seiten = 300;
meinLieblingsbuch.ISBN = "978-3-455-01173-9";
```

Hier wird eine Instanz `Autor` mit einer Instanz `Buch` verknüpft. Diese Bindungsart ist eine **Assoziation**.

Von Feldern zu Eigenschaften (Properties)

Felder sind die einfachste Art, Daten zu speichern - aber riskant: Der Anwender der Klasse hat entweder vollen (`public`) oder gar keinen (`private`) Zugriff. Oft wünscht man sich mehr Kontrolle: nur lesen, Werte vor dem Speichern prüfen, oder beim Ändern weitere Aktionen auslösen.

Dafür gibt es **Eigenschaften (engl. Properties)**. Eine Eigenschaft besitzt eine kleine Methode zum Lesen (Getter) und eine zum Schreiben (Setter):

```
class Land
{
    private string iso3;           // interne Variable (Backing-Field)

    public string ISO3           // öffentliche Eigenschaft
    {
        get                     // Getter zum Lesen
        {
            return iso3;
        }
        set                     // Setter zum Schreiben
        {
            iso3 = value;        // value = der zugewiesene Wert
        }
    }
}
```

Die effektive Speicherung übernimmt eine `private` Variable, die von aussen nicht erreichbar ist. Die Eigenschaft `ISO3` ist `public` und hat denselben Datentyp. Im Setter liefert das Schlüsselwort `value` den zugewiesenen Wert. In der Anwendung verhält sich die Eigenschaft wie ein Feld:

```
Land land = new Land();
land.ISO3 = "DEU";
Console.WriteLine("Der Landcode heisst " + land.ISO3);
```

Auto-Property (Kurzform): Wenn der Getter/Setter nur einen Wert durchreicht, schreibt man kurz `public string Name { get; set; }`. C# legt das Backing-Field automatisch im Hintergrund an. Das ist die häufigste Schreibweise im Alltag.

Tipp: `prop + Tab + Tab` erzeugt eine Auto-Property, `propfull + Tab + Tab` eine vollständige Eigenschaft mit privater Variable (Code Snippets in Visual Studio).

Der Mehrwert von Eigenschaften

Richtig eingesetzt bieten Eigenschaften mehr als Felder:

Read-Only (nur lesbar) - nur ein Getter:

```
private DateTime letzteAktualisierung;

public DateTime LetzteAktualisierung
{
    get { return letzteAktualisierung; }
}
```

Write-Only - analog nur ein Setter (selten gebraucht).

Berechnete Werte - der Getter gibt nicht nur einen gespeicherten Wert zurück, sondern berechnet ihn:

```
public int AnzahlTageSeitAktualisierung
{
    get
    {
        return (DateTime.Now - letzteAktualisierung).Days;
    }
}
```

Die Berechnungslogik bleibt so im Objekt und muss nicht vom Anwender der Klasse durchgeführt werden.

Datenvalidierung - der Setter lässt nur gültige Werte zu:

```
private int einwohnerzahl;

public int Einwohnerzahl
{
    get { return einwohnerzahl; }
    set
    {
        if (value < 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(value), "Einwohnerzahl darf nicht negativ
sein.");
        }
        einwohnerzahl = value;
    }
}
```

Ausblick G2: Dass private Felder gezielt über geprüfte Eigenschaften zugänglich gemacht werden, ist der Kern des Prinzips **Kapselung**. Auch sogenannte Trigger (Folgeaktionen im Setter, z.B. Logging oder das automatische Nachführen eines Datums) gehören dazu. Beides vertiefst du in G2 zusammen mit den Methoden.

Beispiel mit Lücken (Einstiegshilfe)

Lernpfad B: Ergänze die mit ____ markierten Stellen selbst. Die vollständige Version steht oben im Kerninhalt als Kontrolle.

```
internal class Buch
{
    public string Titel;
    public ____ Autor;           // Datentyp ergänzen
    public int Seiten;
    // weiteres Feld ergänzen: ISBN
}

static void Main(string[] args)
{
    Buch meinBuch = ____ Buch();    // Objekt erzeugen
    meinBuch.Titel = "Microserfs";
    meinBuch.____ = "Douglas Coupland"; // Feld setzen
    Console.WriteLine(meinBuch.Titel);
}
```

Aufgaben

Bearbeite die Basisaufgaben verbindlich. Vertiefung und Challenge sind für mehr Tiefe bzw. für Schnelle gedacht. KI darfst du nutzen - zum Erklären und Prüfen, nicht zum blinden Kopieren.

Basis (Pflicht)

Aufgabe 1 - Klasse Buch. Baue die Klasse Buch nach, ergänze ein bis zwei selbst gewählte Felder, erstelle zwei Instanzen mit unterschiedlichen Werten und gib ihre Daten direkt auf der Konsole aus (Feldzugriff).

Aufgabe 2 - Fallbeispiel «Zoo». Bilde einen Zoo mit Klassen und Feldern ab. Wo mehrere Instanzen nötig sind, verwende Listen. Beispiel:

```
internal class Kaefig
{
    public List<Tier> Tiere;
    // weitere Felder
}
```

Erzeuge von jedem Objekttyp mindestens zwei Instanzen und weise sie zu.

Vertiefung

Aufgabe 3 - Eigenes Fallbeispiel. Entwirf ein eigenes Beispiel aus mindestens vier Klassen. Skizziere es zuerst von Hand oder mit draw.io, dann programmiere es. Lege die Skizze im Projekt im Ordner Dokumentation ab. Erzeuge von jedem Typ mindestens zwei Instanzen. Erstelle zum Abschluss per Reverse Engineering ein UML-Klassendiagramm (voller Datenumfang) und speichere einen Screenshot im selben Ordner.

Aufgabe 4 - Eigenschaften statt Felder. Baue dein Fallbeispiel aus Aufgabe 3 so um, dass die Felder zu Eigenschaften werden. Verwende dabei mindestens je einmal: eine Read-Only-Eigenschaft, einen berechneten Wert und eine Datenvalidierung.

Aufgabe 5 - Konsolen-Erfassung. Erweitere dein Programm so, dass der Benutzer die Werte für eine Instanz über die Konsole eingibt (Console.ReadLine). Gib die erfasste Instanz anschliessend wieder auf der Konsole aus. Tipp: Zahlen musst du umwandeln (z.B. int.Parse). Eine Instanz genügt - mehrere verwalten lernst du später mit Sammlungen (X4).

Challenge (PRO, nicht prüfungsrelevant)

Aufgabe 6 - byValue / byReference. Sage zuerst die Ausgabe voraus, teste sie dann:

```
int x = 10;
int y = x;
x = 20;
Console.WriteLine(y);           // Vorhersage? ____

Buch buch1 = new Buch { Titel = "C# Programmierung" };
Buch buch2 = buch1;
buch1.Titel = "C# - Fortgeschritten";
Console.WriteLine(buch2.Titel); // Vorhersage? ____
```

Erkläre den Unterschied: Werttyp (int) gegenüber Referenztyp (Buch).

Aufgabe 7 - JSON-Speicherung. Speichere deine Objekte mit System.Text.Json als JSON-Datei und lies sie wieder ein. Achte bei Assoziationen darauf, Daten nicht redundant zu speichern (denke an Datenbanken). Teste die Wiederherstellung.

```
string jsonString = JsonSerializer.Serialize(buch1);  
File.WriteAllText("buch.json", jsonString);  
  
string gelesen = File.ReadAllText("buch.json");  
Buch buchAusDatei = JsonSerializer.Deserialize<Buch>(gelesen);
```

KI-Prompts

KI ist in diesem Modul ein Pflicht-Werkzeug. Nutze diese Start-Prompts zum Erkunden, Erklären und Prüfen - und beurteile die Antworten kritisch:

- › **Erklären:** «Erkläre mir den Unterschied zwischen einem Feld und einer Eigenschaft (Property) in C# an einem einfachen Alltagsbeispiel.»
- › **Erkunden:** «Wann nutze ich eine Auto-Property { get; set; } und wann eine vollständige Property mit privatem Backing-Field? Nenne je ein Beispiel.»
- › **Prüfen (Gegencheck):** «Hier ist meine Klasse: [Code einfügen]. Entspricht die Namensgebung der Clean-Code-Regel (PascalCase)? Sind die Zugriffstypen sinnvoll gewählt?»
- › **Vertiefung:** «Erkläre an einem Beispiel, warum sich $y = x$ bei einem Werttyp (int) anders verhält als bei einem Referenztyp (Klasse).»

Wichtig: Halte deine KI-Nutzung im KI-Protokoll (Selbsteinschätzungs- bzw. Planungsblatt) fest: Prompt, was war hilfreich, was war falsch oder irreführend?

Weiterführende Informationen

Die folgenden Links vertiefen das Thema. Sie sind freiwillig, aber für deinen Leistungsnachweis nützlich.

Weiterführend

- › Zugriffstypen (Microsoft Learn): <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/accessibility-levels>
- › JSON erklärt: <https://www.derinformatikstudent.de/json/>
- › JSON validieren / formatieren: <https://jsonformatter.org>
- › JSON in C#-Klassen umwandeln: <https://json2.net/json-to-csharp.html>
- › Werttyp vs. Referenztyp und Garbage Collection (für Neugierige): <https://ericlippert.com/2015/05/18/when-everything-you-know-is-wrong-part-one/>

Reflexion

Fülle P3 (Abschluss) deines Themen-Planungsdokuments aus. Kurz zusammengefasst:

Was kann ich jetzt, was ich vorher nicht konnte?

Was war schwierig, und wie habe ich es gelöst?

Wo hat mir KI geholfen, wo hat sie mich in die Irre geführt?